

软件工程

平时作业

分组与选题

□分组

- 2-3人一组
- 自由组队
- 选定组长，合理分工

□选题

- 按自己的兴趣自设
- 项目的功能可根据小组的能力，进行创新和增删
 - 功能不在于多，在于有价值，在于创新！
- 进行技术和进度可行性分析
 - 编程语言没有强制要求，可自选，建议选至少50%的小组成员掌握的语言
 - 需学习的新技术（数据库、前端框架、后端框架、深度学习模型等）不宜太多，如限于2项

软件开发的要求

- 过程：采用迭代的开发过程，覆盖需求、设计、编码到测试各软件开发环节
- 设计：模块化设计，合理的架构风格和设计模式，能支持功能需求和非功能需求，能应对需求变更
- 编程：核心编程语言采用面向对象语言(Java、C++、Python等)，符合Google编程规范
 - 英文：<https://github.com/google/styleguide>
 - 中文：<https://github.com/zh-google-styleguide/zh-google-styleguide>
- 测试：采用xUnit工具进行单元测试，采用人工方式进行系统功能测试，鼓励进行自动化的系统性能测试
- 版本管理：推荐采用Git进行文档、模型和代码的版本管理
 - Github：<https://www.github.com/>
 - Github Guides: <https://guides.github.com/>
 - 或者自部署gitea

项目开发计划

2次作业，对应2个迭代：【此为软件工程课程平时成绩来源】

- 1) hw1：界面原型迭代，应对需求风险
- 2) hw2：技术原型迭代，应对技术风险

优先级高的需求先完成，以应对进度风险。

若时间来不及，抛弃优先级低的需求，不能牺牲质量。

1) hw1 界面原型迭代，应对需求风险

①制定本迭代的《迭代计划》

- 小组分工、进度安排。采用滚动式规划方法。

②调研、分析和定义需求

- 《Vision文档》
- Use case模型：Use case 图和主要Use case的用例规约
 - 包含在《软件需求规约》中

③初选语言、工具和框架，并学习新技术

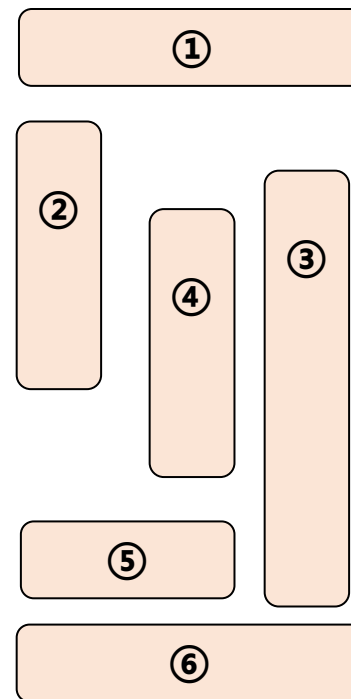
④进行界面设计，实现界面原型

⑤小组内部评审和改进需求文档和界面原型

⑥编写《迭代评估报告》

- 评审记录、开发总结

注：推荐用Git来管理开发中的各文档、模型和代码



界面原型的开发

□ 界面原型

- 界面原型要包括：主要界面和界面上的主要元素

□ 开发界面原型的几种方法

- 图纸（在纸上手绘）
- 位图（Visio等绘图工具绘制并导出）
- 可执行代码，即交互式的界面原型
 - 直接编程，如用HTML或编程语言
 - 用界面原型设计工具，如Axure (<https://www.axure.com>)、Sketch、Zeplin、Figma、.....
 - AI 界面设计工具【text → 可编辑 UI 设计稿，常导出到 Figma 精修】：framer ai、uizard、google stitch、.....
 - ▶ 国内：墨刀modao、mastergo莫高、.....
 - AI 界面&原型生成工具【text-to-code UI】：v0.app、replit、bolt.new【开源自托管版本为 bolt.diy】、.....

软件界面checklist

- ❑ 所选用的人机交互方式是否合理，是否有创新？
- ❑ 每个界面是否都有系统名称或 logo？
- ❑ 界面是否美观？各界面的风格是否一致？
- ❑ 中文还是英文，或两者都支持？
- ❑ 用户是否需要培训？操作是否方便？
- ❑ 是否有在线帮助或提示？
- ❑ 界面大小能否自适应调整？
- ❑ 布局、色彩、字体大小是否合理？
- ❑ 界面是否简洁明了？
- ❑ 用户的操作都有反馈信息吗？
- ❑ 出错信息的说明是否清晰？是否有防错措施？
- ❑ 是否从用户的角度去思考？软件是否越用越好用？

按此检查软件界面

hw1 界面原型迭代的成果要求

□在hw1目录下，组织本次迭代的成果，包括：

- ① 本次迭代计划【模板-迭代计划.简.docx】
- ② Vision文档【模板-vision前景文档.简.docx】
- ③ 软件需求规约SRS【模板-软件需求规约SRS.简.docx】
 - 用例模型：要包含能支撑起主要需求的核心用例，用例图用 plantuml 绘制，截图放在SRS中
 - 用例规约：参考用例规约模板【模板-用例规约.docx】，编写各核心用例的规约。
 - 界面原型【可以有代码，也可以只是界面截图。若有代码或prompt，不要放SRS，放文件 界面原型.docx 中】：
 - ▶要有主要界面、各界面要标明主要界面元素
 - ▶要说明界面、界面元素 和 需求/用例间的关系
 - ▶若使用ai生成界面，还需给出prompt
- ④ 迭代评估报告【模板-迭代评估报告.docx】

□注：绘图使用plantuml，截图后贴在文档中

2) hw2 技术原型迭代，应对架构风险

①制定本迭代的《迭代计划》

②技术方案设计【虽然没有显示提及，但要完成架构视图，是需要有分析模型支撑的】

- 选择架构风格，确定语言、框架、工具
- 设计多个架构视图
- 设计关键算法(若需要)，放在架构文档中
- 撰写和评审软件架构文档
- 选定或撰写编程规范(如google Java编程规范)

③实现技术原型

- 搭建软件架构, 按编程规范实现2 - 3个核心用例

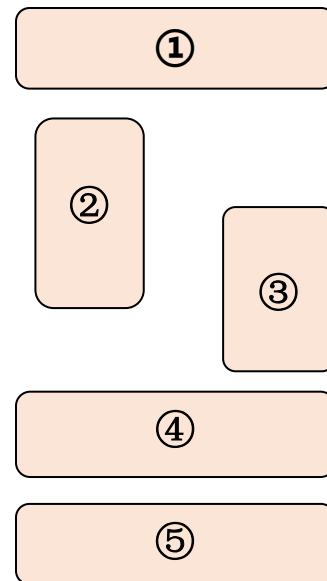
④测试技术原型

- 测试是否达到需求中预定的技术要求，架构是否可行，并根据测试结果进行改进

⑤编写《迭代评估报告》

- 评审和测试记录，开发总结

□ 注：可能对前1个迭代的工作或工件进行返工



hw2 技术原型迭代的成果要求

□在hw2目录下，组织本次迭代的成果，包括：

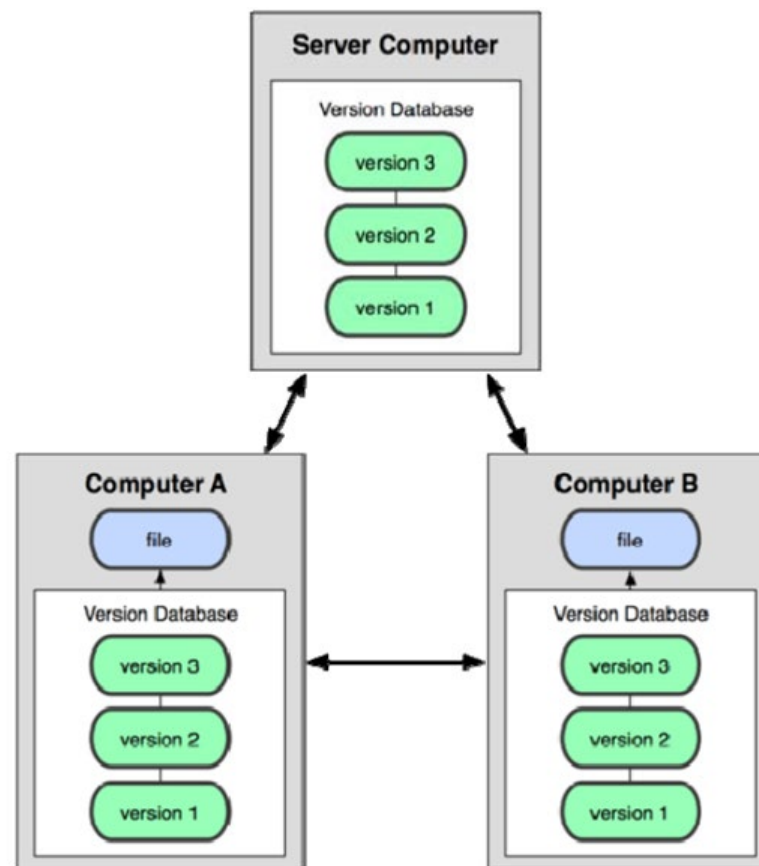
- ① 本次迭代计划【模板-迭代计划.简.docx】
- ② 本次迭代更新后的Vision文档【模板-vision前景文档.简.docx】
- ③ 分析建模【放 分析模型.docx 中】
 - 概念模型：给出系统核心概念类图
 - 分析模型：
 - ▶ 对更新后的用例模型，识别每个核心用例的至少一个用例实现
 - ▶ 每个用例实现应有相应的顺序图、通信图、VOPC类图，三者都得有
 - ▶ 合并各VOPC类图生成整体类图
- ④ 软件架构文档【模板-软件构架文档.docx】
 - 从整体类图中将类组织到包/模块时注意粒度。逻辑架构的粒度要略细些，一般是两级，即把包再细分为子包。
- ⑤ 编程规范
- ⑥ 技术原型的代码【注意不是整个软件的代码，完整代码在学期综合实践中提交】
- ⑦ 迭代评估报告

□注：绘图使用plantuml，截图后贴在文档中

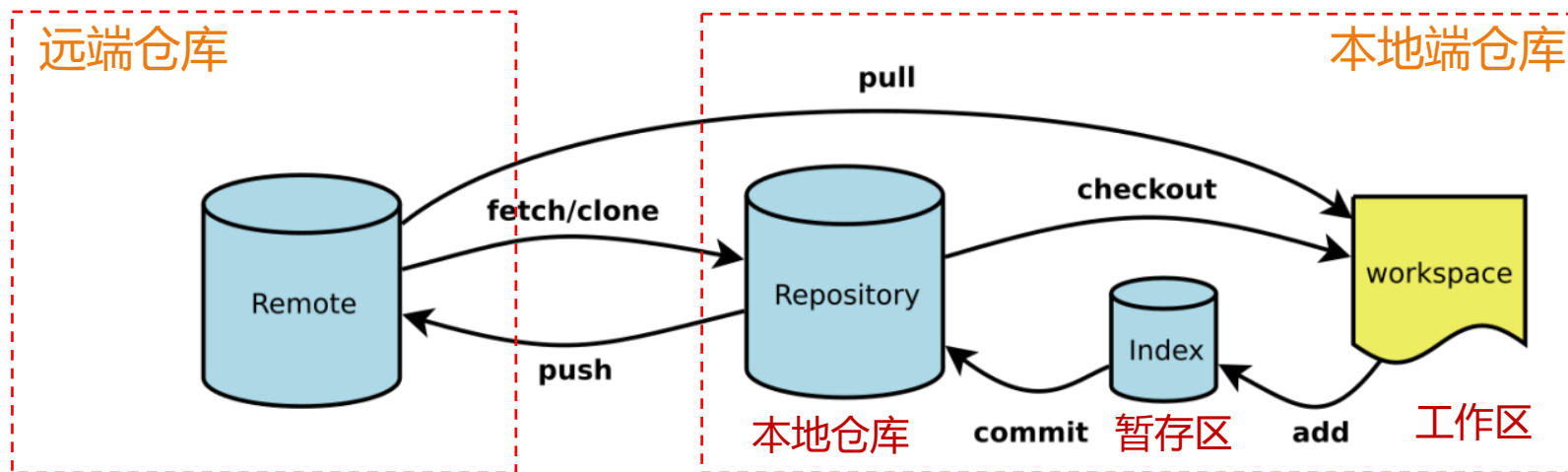
分布式版本控制系统

- 适合于任何规模的团队开发
- 支持Internet或局域网等各种网络
 - 有一个中央仓库
 - 开发前在本机上拷贝一个完整软件仓库
 - 开发完把自己工作提交到本地仓库中
 - 需要同步给协作者时再递交到中央仓库
 - 版本库分步存储于各协作者电脑中

□ 典型系统： **Git**

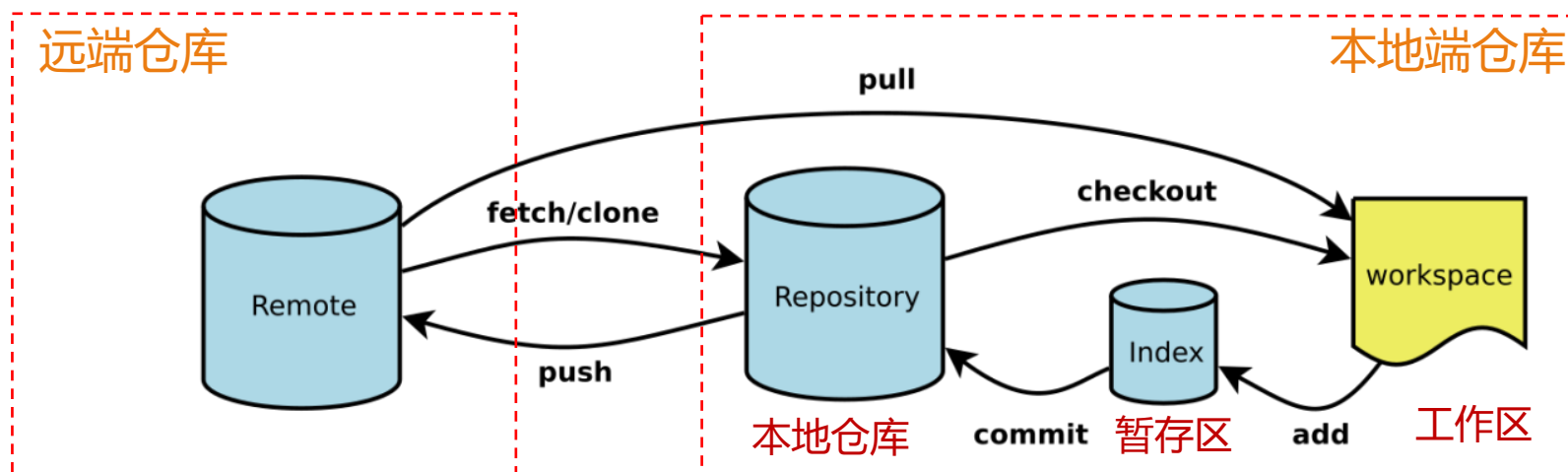


Git 的仓库组成



- **Workspace**: 工作区，是你平时存放项目代码的地方
- **Index / Stage**: 暂存区，用于临时存放你的改动，事实上它只是一个文件，保存即将提交到文件列表信息
- **Repository**: 本地仓库，存放着你提交的所有版本的数据
- **Remote**: 远程仓库，托管代码的服务器

Git 的常用命令



- ❑ **fetch/clone**命令：将中央服务器上该项目的远程仓库拷贝到本地机器上
- ❑ **pull**命令：将中央服务器上远程仓库的所有最新提交全部拉取到本地仓库并进行合并
- ❑ **push**命令：将本地仓库中的本地提交推送到中央服务器的远程仓库中
- ❑ **commit**命令：将暂存区中的文件提交到本地仓库
- ❑ **checkout**命令：将(改乱的)文件重置为暂存区或者本地仓库中的文件内容
- ❑ **add**命令：将指定的文件（更改）保存到暂存区

Git中的提交 (Commit)

- 原子性：提交粒度上要确保每次提交只做一件事情，不要把多件事混在一次提交中
- 在commit命令中指定该提交的描述消息
 - 类型：新功能、缺陷修复、重构、测试、文档、格式化等
 - 主题：提交的简要描述
 - 主体：提交的详细描述
 - 链接：与其他软件制品（如开发任务、缺陷）的链接关系

fix: couple of unit tests for IE9

Older IEs serialize html uppercased, but IE9 does not...

Would be better to expect case insensitive, unfortunately jasmine does not allow to user regexps for throw expectations.

Closes #392

Git 多人协同工作流1: Github Flow

□主分支: Master分支

□短分支: 开发新功能或修复缺陷时, 创建一个新分支, 完成后通过Pull Request (PR) 合并至Master, 删除分支

[GitHub flow Docs](#)



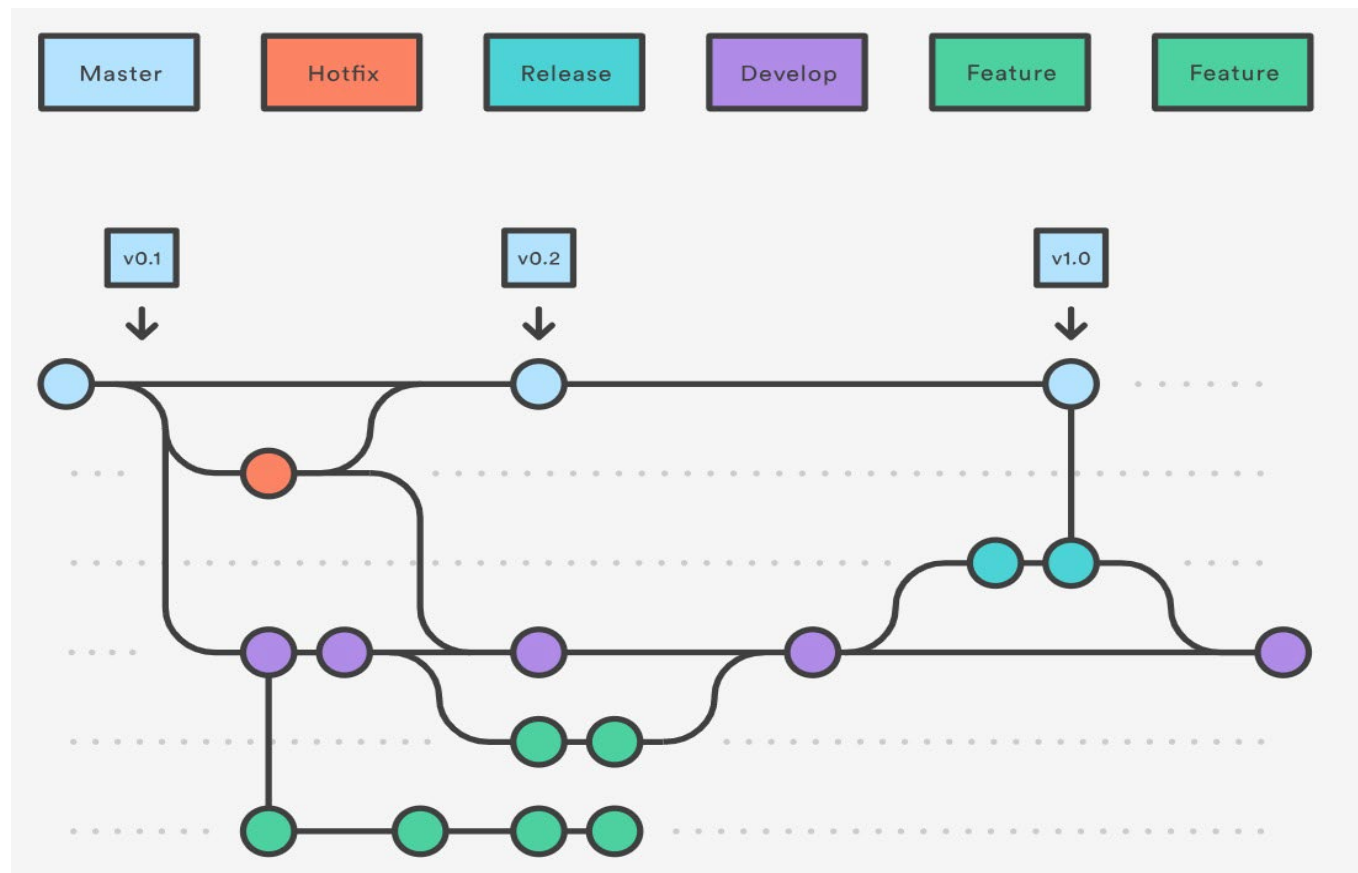
Git 多人协同工作流2: Gitflow Workflow

□ 两个主分支:

- Master分支保存官方的发布历史
- Develop分支用于集成各种功能开发的分支

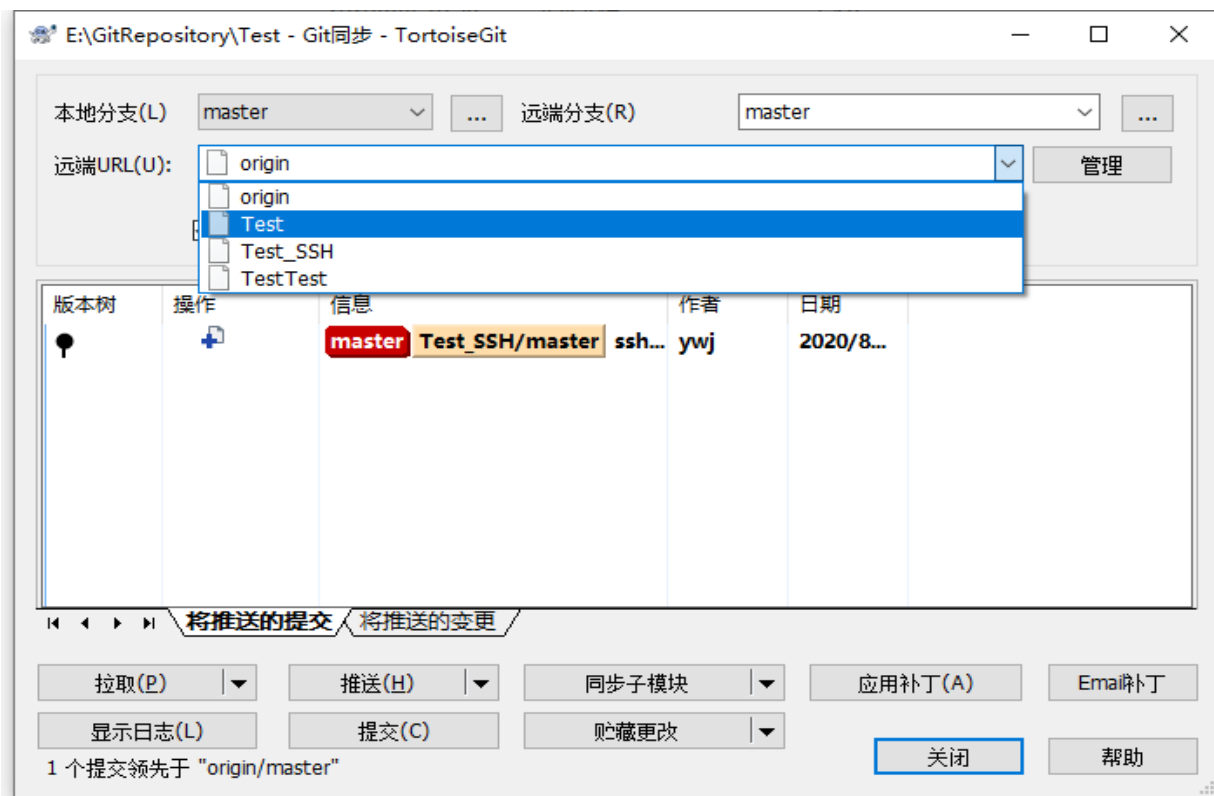
□ 在创建新的功能开发分支时, 父分支应该选择Develop

[Gitflow Workflow Tutorial](#)



Git客户端软件 TortoiseGit

- 一种开源的Git版本控制系统客户端软件
 - 它采用Windows图形化界面，使得使用Git变得更加简便和易于操作
- 安装访问TortoiseGit官网



Git 学习资料

- ❑ GitHub Guides: <https://guides.github.com/> (必用)
 - Video: https://www.bilibili.com/video/BV1i44y1e7hv/?spm_id_from=333.337.search-card.all.click&vd_source=c2fea6caa8dabef148d703c539e441c8
- ❑ GitHub Flow: <https://docs.github.com/cn/get-started/quickstart/github-flow> (简单, 建议现在使用)
 - Video: https://www.youtube.com/watch?v=juLlxo42A_s
- ❑ Gitflow Workflow: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow> (高级, 建议以后使用)
 - Video : <https://www.youtube.com/watch?v=8UguQzmswC4>
- ❑ Conventional Commits: <https://www.conventionalcommits.org/en/v1.0.0/>
- ❑ github/gitignore: <https://github.com/github/gitignore>